CSE 221: Algorithms

Chapter 0 — Introduction to Algorithms

Prepared by Ariyan Hossain [AYO]

1 What is an Algorithm?

An **algorithm** is a procedure to accomplish a specific task. It takes any possible input instance and transforms it into the desired output through a sequence of computational steps.

Example: Preparing an omelet

To prepare an omelet, we can follow steps like:

1) Get the frying pan.
2) Get the oil.
    a) Do we have oil?
        i) If yes, put it in the pan.
        ii) If no, do we want to buy oil?
            1. If yes, then go out and buy.
            2. If no, we can terminate.
3) Turn on the stove, etc.

1.1 How do we judge an algorithm?

There are two main criteria for judging the merits of algorithms:

- **Correctness:** Does the algorithm solve the problem in a finite number of steps?
  An algorithm for a computational problem is **correct** if, for every valid input instance, it halts (finishes) in finite time and outputs the correct solution. To prove correctness, the problem must be defined precisely:

  1) what inputs are allowed, and
  2) what the output must satisfy.

  You can prove an algorithm is incorrect by finding a good counterexample.

- **Efficiency:** How many resources (memory and time) does it take to execute?
  We focus on how resource requirements grow as the input size increases, rather than exact running times on a specific computer. An algorithm may be considered inefficient if another algorithm solves the same problem using fewer resources, especially for large inputs.

Formal algorithm definition

A **finite** set of statements that guarantees an optimal solution in a finite interval of time.

2 Examples of real-life algorithms

i) The internet allows people around the world to access huge amounts of information. Algorithms such as **Dijkstra's Algorithm** help route data efficiently (finding good paths), and search engines use algorithms to quickly find pages that contain specific information.

ii) File compression: when you compress a large text file to occupy less space, algorithms such as **Huffman coding** allow data to be stored using fewer bits.

3 Algorithm specification

A well-specified algorithm has the following properties:

- **Input:** Every algorithm must take zero or more input values.
- **Output:** Every algorithm must produce an output as a result.
- **Definiteness:** Every statement/instruction must be unambiguous (only one interpretation).
- **Finiteness:** The algorithm must produce results within a finite number of steps for all valid cases.
- **Effectiveness:** Every instruction must be basic enough to be carried out and feasible in practice.

3.1 Resource analysis

To analyze algorithms, we want to predict the amount of resources required:

- **Memory:** how much space is needed?
- **Computational time:** how fast the algorithm runs (number of primitive steps).

If computers were infinitely fast and had infinite memory, any correct method for solving a problem would work. In reality, we also want implementations to follow good software engineering practice (well-designed and documented). A good algorithm description should allow a competent but skeptical programmer to implement it correctly in their preferred programming language, without needing to understand *why* it works.

4 Describing algorithms

Algorithms are abstract, precise procedures, not programs themselves. Programs are concrete implementations of algorithms in a specific programming language.

Plain English alone is often too vague, while real code can be too detailed and rigid for explanation. A common way to describe algorithms is **pseudocode**, which combines the structure of programming and mathematics with clear language to express each step precisely.

Pseudocode: `FindMaximum`

Algorithm `FindMaximum`
Input: list of numbers L
Output: largest number in L

`max` $\leftarrow L[0]$
for each number x **in** L **do**
    if $x >$ `max` **then**
        `max` $\leftarrow x$
    end if
end for
return `max`
End Algorithm

5 Additional notes

- Even if you never have to explain your algorithms to anyone else, it is still important to develop them with an audience in mind. A major job of an algorithm designer is teaching other people how and why algorithms work.
- Algorithmic problems are often presented in standard English using real-world objects. As algorithm designers, we restate these problems in terms of formal, abstract, mathematical objects such as numbers, arrays, lists, graphs, and trees so that we can reason about them precisely.